

Stylometric Epoch Classification of German Prose

Final project report – Advanced Python for NLP

Joel Dick

B.A. Computerlinguistik, Heinrich-Heine-Universität Düsseldorf

October 2026

Abstract

This project asks how well the literary epoch of a German prose text can be predicted from four simple stylometric features: median sentence length, standardised type-token ratio, function word ratio and the density of distinct characters. A corpus of 268 texts by 192 authors was collected from Project Gutenberg and labelled as *Goethezeit*, *Realismus* or *Moderne* by the death year of the author. Features were extracted with spaCy, and four classifiers were evaluated on held-out authors. The best model, a nearest-centroid classifier, reaches a macro-F1 of 0.44 on the test set, compared with 0.21 for a majority baseline; its accuracy (0.44) does not exceed the baseline's. Sentence length and the function word ratio carry almost all of the signal, while vocabulary richness and character density add little or nothing. Literary epochs leave a measurable but weak trace in these surface features, and many errors can be traced back to the death-year labels and to individual authors' styles.

1 Introduction

Literary history divides German literature into epochs such as *Klassik*, *Romantik*, *Realismus* or *Moderne*. These labels are usually justified by content, themes and aesthetic programmes. Stylometry, the quantitative study of style, asks whether such categories are also visible on the surface of the text: in how long the sentences are, how varied the vocabulary is, or how many grammatical function words a writer uses.

This project turns that question into a supervised classification task:

How well can a classifier predict the literary epoch of a German prose text from four stylometric features?

Framing the question as classification has two advantages over a purely descriptive comparison of epochs. First, it gives a clear measure of success (accuracy and F1 on unseen texts) that can be compared with a baseline. Second, it forces an honest evaluation: if the features only seemed to differ between epochs because the same authors appear again and again, a classifier tested on *new* authors will reveal this.

The scope is deliberately small. The project uses exactly four interpretable features instead of hundreds of word frequencies, so that every result can be explained linguistically. The contributions are (1) a reproducible pipeline from the Gutenberg catalog to the evaluation, (2) an author-disjoint evaluation that quantifies how much author leakage would inflate the scores, and (3) a feature-level and text-level analysis of what the classifiers learn and where they fail.

2 Related work

Stylometry has its roots in authorship attribution. Mosteller and Wallace (1964) used the frequencies of function words to decide the authorship of the disputed *Federalist Papers*, and

Burrows (2002) proposed *Delta*, a distance between standardised word frequencies that is still a standard method; this project’s nearest-centroid classifier follows the same idea of a standardised “fingerprint”. Stamatatos (2009) surveys the feature types used in the field, from lexical measures such as sentence length and vocabulary richness to syntactic features derived from part-of-speech tags. Function words are considered particularly useful because they are frequent, largely independent of topic and used unconsciously.

That stylometric signals also reflect literary history, and not only individual authors, was shown for German by Jannidis and Lauer (2014), who found that Delta distances between German novels of the long nineteenth century group texts partly by period. Vocabulary richness measures are known to depend on text length; standardised or moving-average type-token ratios (Covington and McFall, 2010) and MTLT (McCarthy and Jarvis, 2010) were proposed to solve this. Counting characters with named entity recognition follows work on social networks in novels (Elson et al., 2010).

3 Data

3.1 Source

All texts come from Project Gutenberg (Project Gutenberg, 2026), the largest collection of public-domain ebooks. Gutenberg was chosen because it is freely available, contains a large number of German literary texts from the eighteenth to the early twentieth century, and provides a machine-readable catalog (`pg_catalog.csv`) with authors, their life dates, languages and Library of Congress classifications. Its drawbacks are discussed in Section 8: the collection is not a representative sample of German literature, publication dates are missing, and genre metadata are sparse.

3.2 Filtering the catalog

The catalog contains one row per ebook. The script `src/catalog.py` applies the filters listed in Table 1. Two decisions are worth explaining. The Library of Congress class *PT* (German literature) is used as the main genre filter because it is available for almost all German books, whereas the subject field is often empty; it also removes German translations of foreign literature, whose style would reflect the translator. Since PT also contains drama and poetry, a keyword list (e.g. *Drama*, *Trauerspiel*, *Gedichte*, *Briefe*) removes books whose title, subjects or bookshelves indicate a non-prose genre. The list is part of `config.yaml`.

Table 1: Filtering of the Gutenberg catalog (numbers of books left after each step, printed by `python main.py -step catalog`).

Filter step	Books
Full catalog	79,474
Type <i>Text</i> and language <i>de</i>	2,404
Library of Congress class PT (German literature)	1,201
No translator listed	1,166
No drama/poetry/non-fiction keywords	787
Author death year inside an epoch	688
Duplicate editions removed	687

3.3 Epoch labels

Gutenberg does not record when a work was first published; the field *Issued* is the date of the digitisation. The epoch is therefore derived from the death year of the author, which is parsed from the *Authors* field with a regular expression (e.g. *Fontane, Theodor, 1819-1898*). A text is labelled *Goethezeit* if its author died up to 1860, *Realismus* if the author died between 1861 and 1910, and *Moderne* if the author died between 1911 and 1960. The boundaries are shifted later than the usual textbook dates of the epochs because authors die some time after their productive period: Theodor Storm (d. 1888) and Theodor Fontane (d. 1898) are correctly labelled *Realismus*, Franz Kafka (d. 1924) and Arthur Schnitzler (d. 1931) *Moderne*. *Goethezeit* is used as an umbrella term for Aufklärung, Klassik and Romantik, because each of these alone has too few prose texts on Gutenberg for a separate class. This labelling is a proxy and introduces noise; it is discussed in Section 8.

3.4 Corpus construction

The candidate list is shuffled with a fixed seed. The download script (`src/download.py`) walks through it and keeps a text if its author has fewer than three texts in the corpus, its epoch has fewer than 120 texts, and the cleaned text has at least 15,000 words. The per-author cap prevents prolific authors from dominating a class, the per-epoch cap keeps the classes balanced, and the minimum length guarantees that every text can provide a sample of the required size.

Every text is cleaned (`src/cleaning.py`): the Gutenberg license header and footer are removed, editorial notes in square brackets and emphasis markup are deleted, and hard-wrapped lines are joined into paragraphs. From each text a contiguous passage of exactly 10,000 words is taken, starting after the first 10 % of the text in order to skip title pages, prefaces and tables of contents. Using a sample of fixed size removes text length as a confounding variable and keeps the processing time manageable.

Table 2 shows the resulting corpus. The 687 candidates were unevenly distributed (79 Goethezeit, 175 Realismus, 433 Moderne). Only *Moderne* reached the cap of 120 texts; for the two older epochs, every candidate that passed the length threshold and the author cap was used. The corpus is therefore imbalanced, with *Moderne* as the largest class (45 %). Most authors are represented by a single text: 140 of the 192 authors have one text, 28 have two and 24 have three.

Table 2: The final corpus.

Epoch (author death year)	Texts	Authors
Goethezeit (≤ 1860)	55	35
Realismus (1861–1910)	93	60
Moderne (1911–1960)	120	97
Total	268	192

4 Method

4.1 Preprocessing

Every sample is processed with the German spaCy pipeline `de_core_news_lg` (Honnibal et al., 2020), which provides tokenisation, sentence segmentation (from the dependency parser), universal part-of-speech tags and named entities. All features are computed on *words*, i.e. tokens that are neither punctuation nor whitespace.

4.2 Features

1. Median sentence length. The number of words in each sentence, summarised by the median. The median is preferred to the mean because a few very long sentences or segmentation errors would dominate the mean. The hypothesis is that sentences become shorter from the periodic style of the Goethezeit to the more paratactic prose of the Moderne.

2. Standardised type-token ratio (STTR). The type-token ratio (distinct words divided by words) measures vocabulary richness, but it falls with text length, so raw values are not comparable between texts (Covington and McFall, 2010). The sample is therefore cut into consecutive windows of 1,000 words, the TTR of every window is computed on lower-cased word forms, and the values are averaged.

3. Function word ratio. The share of determiners, adpositions and conjunctions (universal POS tags DET, ADP, CCONJ, SCONJ) among all words. Function words are the classic stylometric signal (Mosteller and Wallace, 1964; Burrows, 2002). They are identified by their POS tag instead of a word list, so that historical spelling variants are covered as well. A higher ratio indicates a more nominal, syntactically elaborated style.

4. Character density. The number of distinct characters per 1,000 words, based on spaCy’s PER entities. A character typically appears under several names (“Effi”, “Effi Briest”, “Effi’s”), so names are normalised and merged with transparent rules: titles such as *Herr* or *Gräfin* are removed, genitive forms are merged with their base form, multi-word names are grouped by their last word, and a single name is added to a multi-word name that contains it. Names mentioned only once are discarded to remove one-off recognition errors. The feature approximates the size of the cast, which may grow with the panoramic social novels of Realism.

4.3 Classifiers

Four classifiers are compared, all trained on the four features:

- a **majority baseline** that always predicts the most frequent epoch of the training set;
- a **nearest-centroid classifier**, implemented in NumPy (Harris et al., 2020) for this project (`src/centroid.py`): the features are z-standardised, the mean vector of every epoch is its “fingerprint”, and a text is assigned to the epoch with the closest fingerprint (Euclidean distance) – the idea behind Burrows’ Delta;
- **logistic regression** with standardised inputs, a linear model whose coefficients can be interpreted;
- a **random forest** (Breiman, 2001) with 300 trees, which can model non-linear relations and interactions.

The library models come from scikit-learn (Pedregosa et al., 2011).

4.4 Evaluation design

Author-disjoint split. If texts by the same author occurred in the training and in the test set, a classifier could recognise the author – whose epoch never changes – instead of the epoch. All splits are therefore grouped by author using scikit-learn’s `StratifiedGroupKFold`, which also keeps the epoch proportions similar across folds. About 20% of the authors are held out as a test set that is used exactly once.

Model selection. The four models are compared by five-fold, author-grouped cross-validation on the training set. The model with the highest mean macro-F1 is designated the best model; the test set plays no role in this choice. Macro-F1 (the unweighted mean of the per-class F1 scores) is the main metric, because it is not inflated by a model that ignores a small class.

Measuring author leakage. To show why the grouped split matters, the cross-validation is repeated with ordinary, non-grouped stratified folds, in which texts of one author can end up on both sides.

Feature analysis. Three complementary methods show which features matter: the standardised coefficients of the logistic regression, the permutation importance of the best model on the test set (the drop in macro-F1 when one feature is randomly shuffled), and an ablation study with each feature alone and with all but one feature. Before classification, a Kruskal-Wallis test (Kruskal and Wallis, 1952) checks for every feature whether its distribution differs between the epochs at all.

5 Implementation

The project is a Python package with one module per pipeline step and a single entry point, `main.py`. All parameters (paths, URLs, epoch boundaries, keyword lists, sample size, model names, random seeds) are stored in `config.yaml`, so that data and logic are separated; the command-line interface (`argparse`) selects the configuration file and the step to run. Each step writes its result to disk and the next step reads it, which makes steps independently re-runnable and allows interrupted runs to resume. Table 3 gives an overview.

Table 3: Modules of the pipeline and their role.

Step	Module	Role
–	<code>main.py</code>	Entry point; parses the command line and calls the steps in order.
–	<code>config.py</code>	Reads <code>config.yaml</code> and resolves all paths relative to the project root.
catalog	<code>catalog.py</code>	Downloads the Gutenberg catalog, parses author entries with a regular expression, applies the filters of Table 1, writes <code>candidates.csv</code> .
catalog	<code>epochs.py</code>	Maps death years to epochs; counts texts and authors per epoch.
download	<code>download.py</code>	Downloads texts (cached, with a delay between requests), applies the per-author and per-epoch caps and the length threshold, writes <code>corpus_metadata.csv</code> .
download, features	<code>cleaning.py</code>	Removes the Gutenberg boilerplate and markup, joins lines, extracts the 10,000-word sample.
features	<code>dataset.py</code>	Streams the samples through spaCy (<code>nlp.pipe</code> fed by a generator), computes the features and appends one row per text to <code>features.csv</code> .
features	<code>features.py</code>	The four feature functions and the name-merging rules.
analyse	<code>describe.py</code>	Per-epoch statistics and Kruskal-Wallis tests.
analyse	<code>centroid.py</code>	The NumPy nearest-centroid classifier (a class with <code>fit</code> and <code>predict</code>).
analyse	<code>classify.py</code>	Author-disjoint split, cross-validation, test evaluation, feature importance, ablation; saves all tables, <code>metrics.json</code> and the pickled best model.
analyse	<code>plots.py</code>	All figures of this report (Matplotlib, Hunter, 2007).

A few implementation choices deserve mention. The feature step uses a generator (`iter_samples`) that reads, cleans and samples one text at a time, so that complete novels never have to be held in memory together; only the numbers are stored, never spaCy’s `Doc` objects. Each feature

is a small function that depends only on token attributes; this made it possible to unit-test every feature on hand-made `Doc` objects whose correct values can be counted by hand. The test suite (`tests/`, 31 tests run with `pytest`) also covers the catalog parsing, the text cleaning and the classifier. The nearest-centroid classifier inherits `scikit-learn`’s estimator interface, so it is cross-validated with exactly the same code as the library models. Data handling uses `pandas` (McKinney, 2010) and the statistical test `SciPy` (Virtanen et al., 2020).

6 Results

6.1 Feature distributions



Figure 1: Distribution of the four features per epoch. Each dot is one text; boxes show the quartiles.

Figure 1 and Table 4 show that two of the four features change systematically from the oldest to the youngest epoch. The median sentence length falls from 17 words in the Goethezeit to 14 in Realismus and 12 in the Moderne, and the function word ratio falls from 0.298 to 0.294 and 0.285 (epoch means). Both differences are highly significant in the Kruskal-Wallis test ($p < 10^{-10}$ and $p < 10^{-4}$). The direction matches the expectation of a shift from long, hypotactic periods towards shorter, more paratactic sentences with fewer prepositional and conjunctive constructions.

The standardised type-token ratio ($p = 0.22$) and the character density ($p = 0.48$) do not differ between the epochs. Even for the two significant features, the distributions overlap strongly: the spread *within* each epoch (standard deviation of the sentence length: 3.4–4.7 words) is larger than the difference between neighbouring epochs (2–3 words). Every epoch contains texts with short and with long sentences, so no single feature can separate the epochs well.

Table 4: Feature means per epoch (standard deviation in brackets) and Kruskal-Wallis test across the three epochs.

Feature	Goethezeit	Realismus	Moderne	H	p
Median sentence length	17.09 (4.73)	15.33 (4.68)	12.43 (3.44)	48.6	$2.8 \cdot 10^{-11}$
STTR	0.508 (0.031)	0.497 (0.033)	0.504 (0.027)	3.0	0.22
Function word ratio	0.298 (0.021)	0.294 (0.017)	0.285 (0.022)	19.6	$5.6 \cdot 10^{-5}$
Character density	1.57 (0.69)	1.46 (0.58)	1.47 (0.68)	1.4	0.48

6.2 Classification

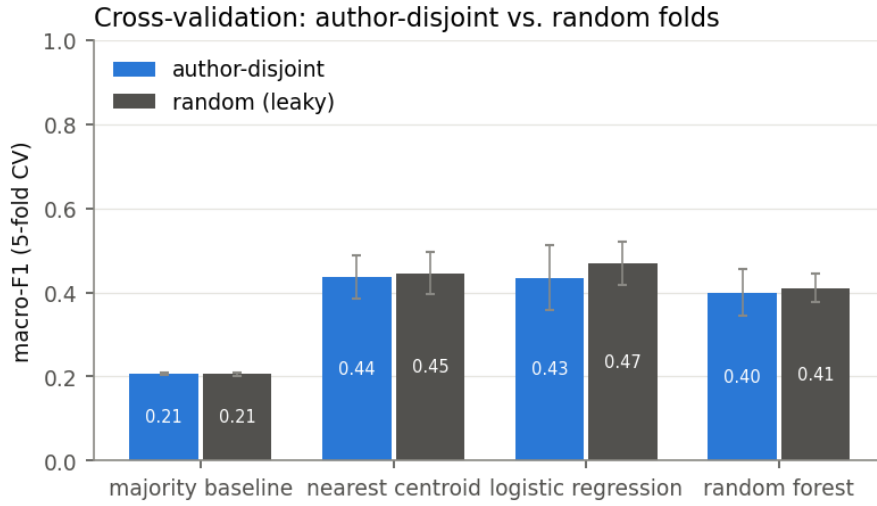


Figure 2: Macro-F1 in five-fold cross-validation on the training set, with author-disjoint folds and with random (leaky) folds. Error bars show the standard deviation across folds.

Table 5: Results on the held-out test set (54 texts by 38 authors not seen in training).

Model	Accuracy	Macro-F1
Majority baseline	0.444	0.205
Nearest centroid (NumPy)	0.444	0.438
Logistic regression	0.426	0.375
Random forest	0.463	0.420

Cross-validation. The training set contains 214 texts by 154 authors. In author-disjoint cross-validation (Figure 2), all three classifiers clearly beat the majority baseline (macro-F1 0.21): the nearest-centroid classifier reaches 0.437 ± 0.051 , logistic regression 0.435 ± 0.077 and the random forest 0.400 ± 0.057 . The differences between the three models are much smaller than their standard deviations across folds; the nearest-centroid classifier was selected as the best model, but this choice is effectively a tie with logistic regression. The simplest model performing as well as the more flexible ones suggests that with four features there is little non-linear structure for the random forest to exploit.

Author leakage. With random, non-grouped folds, the scores rise only slightly (logistic regression: 0.469 instead of 0.435; nearest centroid and random forest: +0.01), well within the

variation between folds. This does not mean that the author-disjoint split is unnecessary. Because of the cap of three texts per author, 140 of the 192 authors have only one text, so there is little author information that could leak. With more texts per author, the gap would be expected to grow.

Test set. Table 5 shows the single evaluation on the 54 held-out texts. The nearest-centroid classifier reaches a macro-F1 of 0.438, twice the baseline’s 0.205, and confirms the cross-validation estimate. Its accuracy (0.444), however, is identical to that of the baseline, which always predicts *Moderne* and is right for 24 of the 54 texts. The two metrics measure different things: the baseline never recognises a Goethezeit or Realismus text, whereas the nearest-centroid classifier is the only model with a balanced recall across all three epochs (0.45, 0.37 and 0.50). Logistic regression and the random forest favour the large *Moderne* class (recall 0.67 and 0.62) at the expense of the Goethezeit (0.18 and 0.27). With only 54 test texts, a single text changes the accuracy by almost two percentage points, so the differences between the three classifiers on the test set should not be over-interpreted.

The confusion matrix of the best model (Figure 3) shows that *Realismus* is the hardest epoch: 11 of its 19 texts are classified as *Moderne*. Of all 30 errors, 22 confuse neighbouring epochs and 8 confuse the Goethezeit with the *Moderne*.

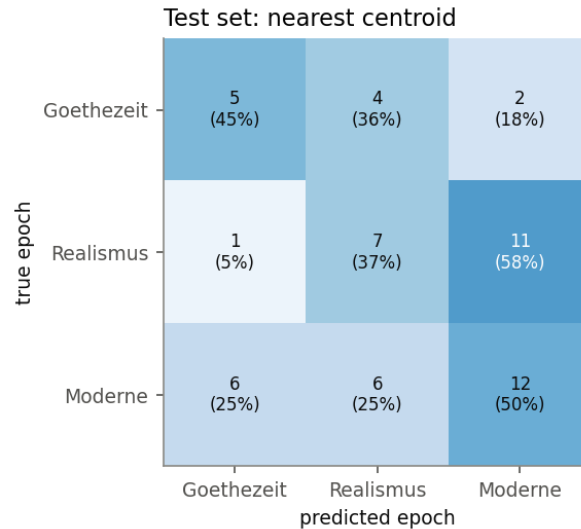


Figure 3: Confusion matrix of the best model on the test set (counts and row percentages).

6.3 Which features matter?

The three methods of feature analysis agree on the main points (Figure 4, Table 6). **Median sentence length** is the most important feature. It has by far the largest logistic regression coefficients (positive for the Goethezeit, +0.53, strongly negative for the *Moderne*, −0.76), it is the best single feature (macro-F1 0.40 on its own), and removing it costs the most (0.09). The **function word ratio** is the second most useful feature on its own (0.36).

The **STTR** is weak on its own (0.24, close to the baseline) but removing it still lowers the score by 0.06, and its permutation importance on the test set is as high as that of the other two features. It therefore seems to help in combination with sentence length rather than on its own; its negative coefficient for *Realismus* reflects the slightly lower lexical diversity of that epoch. The **character density** carries no usable information: alone it scores at baseline level (0.20), its permutation importance is close to zero, and removing it even *improves* the cross-validated

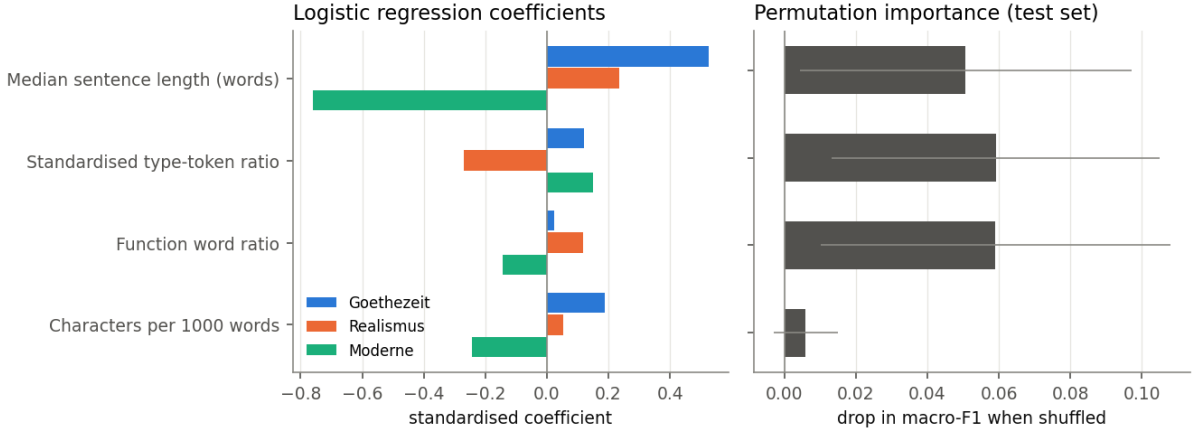


Figure 4: Left: standardised logistic regression coefficients per epoch. Right: permutation importance of the best model on the test set (mean and standard deviation over 30 repetitions).

macro-F1 by 0.04. The feature set was nevertheless not changed after these results, because selecting features on the basis of evaluation results would make the reported scores optimistic.

Table 6: Feature ablation with logistic regression (cross-validated macro-F1 on the training set; all four features: 0.435).

Feature	Only this feature	Without this feature	Drop
Median sentence length	0.397	0.346	+0.089
STTR	0.239	0.376	+0.059
Function word ratio	0.363	0.412	+0.022
Character density	0.201	0.472	−0.037

7 Qualitative analysis

The 30 test texts misclassified by the nearest-centroid classifier (`results/misclassified.csv`) were inspected individually. Most errors fall into four groups.

Label noise from death years. Several errors are caused by the epoch label rather than by the classifier. Leopold Schefer was born in 1784 and belongs to the generation of the Romantics, but he died in 1862 and is therefore labelled *Realismus*; all three of his texts are misclassified. Hermann Heiberg (1840–1910) died exactly at the boundary between Realismus and Moderne, and all three of his test texts are predicted as Moderne. The opposite case is Wilhelm Holzamer (1870–1907): because he died young, his work from around 1900 is labelled Realismus. Its very short sentences (median 7 words) are classified as Moderne.

Individual style versus epoch style. Thomas Mann’s *Buddenbrooks* and *Tonio Kröger* are correctly classified as Moderne, but *Der Tod in Venedig*, known for its long and elaborate sentences, is predicted as Goethezeit (median sentence length 17, exactly the Goethezeit median). Similarly, Josef von Neupauer’s *Oesterreich im Jahre 2020* has a median sentence length of 25 words, far above the Realismus average, and is assigned to the Goethezeit. In both cases the classifier is right about the style and wrong about the epoch: a single author can write in a way that is untypical for their time.

Genre mistakes of the corpus filter. Some texts are not narrative prose although they passed the filter: Theodor von Kobbe’s *Humoristische Erinnerungen* are a memoir, Egon Erwin Kisch’s *Der Fall des Generalstabschefs Redl* is a piece of reportage, and Ludwig Bechstein’s *Märchenbuch* is a collection of fairy tales. All three are misclassified, which shows that genre influences the features at least as much as the epoch.

Errors cluster by author. Five authors (Schefer, Heiberg, Wieland, Dauthendey and Adlersfeld-Ballestrem) account for 12 of the 30 errors. Because texts by the same author have similar features, they tend to be right or wrong together. The test set therefore contains fewer independent observations than its 54 texts suggest, which adds to the uncertainty of the scores.

Overall, the qualitative analysis suggests that the model’s errors are not random. A considerable share of them is explained by the proxy labels, by authors with an untypical style and by texts of the wrong genre.

8 Challenges and open issues

Death year as epoch label. The largest source of error is the label itself. An author’s death year only approximates when a text was written. Long-lived authors who published early are pushed into a later epoch: Ludwig Tieck, a Romantic, died in 1853 and is still labelled correctly, but Paul Heyse (d. 1914) or Friedrich Spielhagen (d. 1911), both Realists, fall into *Moderne*. Epochs are also not sharp temporal categories: Realist and modernist texts were written in the same decades around 1900. A better label would be the year of first publication, which Gutenberg does not provide; the ELTeC collection (Schöch et al., 2021) does record it, but only for novels between 1840 and 1920.

Sparse genre metadata. Many German Gutenberg books have no subject headings, so the prose filter relies on the Library of Congress class and on keywords. Collections that mix prose with poems, travel writing or essays without saying so in the title can slip through.

A skewed collection. Gutenberg reflects what volunteers chose to digitise. Popular fiction of the late nineteenth century is strongly represented, canonical texts of the Goethezeit much less. The corpus is therefore not representative of German literature, and the *Moderne* class is limited to texts that were already in the public domain in the USA, i.e. mostly published before 1930.

NLP tools and historical German. spaCy’s German models are trained on contemporary newspaper text. Older spelling (*Thür, seyn, giebt*) and long, deeply nested sentences are harder for the tagger and the parser, so the error rate of all features is probably higher for older texts – a systematic bias that correlates with the label. Dialogue also affects sentence segmentation, because quoted speech may be split into several sentences or merged with the narrative.

Character counting. Named entity recognition confuses characters with places or with each other, and the merging rules are deliberately simple: two characters with the same surname (father and daughter) are counted as one, and nicknames are not linked to full names. Character density should thus be seen as a rough approximation of the size of the cast.

Small data. With a few hundred texts, the test set contains only a few dozen texts. Single misclassifications change the scores by several percentage points, so differences between models should not be over-interpreted; the standard deviations across cross-validation folds (Figure 2) give an impression of this uncertainty.

Open issues. Natural extensions are publication-year labels, more features (e.g. part-of-speech n-grams or the share of direct speech), a regression on the year instead of discrete epochs, and a comparison with Burrows’ Delta on the most frequent words.

9 Conclusion

The research question was how well a classifier can predict the literary epoch of a German prose text from four stylometric features. The answer is: better than chance, but not well. On texts by unseen authors, the best classifier reaches a macro-F1 of 0.44 against 0.21 for a majority baseline, while its accuracy does not exceed the baseline’s. The signal comes almost entirely from sentence length and the function word ratio, which both decrease from the Goethezeit to the Moderne; vocabulary richness contributes little and the number of characters nothing. A simple nearest-centroid “fingerprint” in NumPy performs as well as logistic regression and a random forest.

The qualitative analysis shows that a considerable part of the remaining error is not a failure of the classifier but of the setup: death years are a noisy proxy for the epoch of a text, individual authors deviate from the style of their time, and the genre filter lets some non-narrative texts through. The most promising improvements are therefore better labels (publication years), a stricter genre selection and richer syntactic features, rather than more complex classifiers.

References

- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- John Burrows. ‘Delta’: a measure of stylistic difference and a guide to likely authorship. *Literary and Linguistic Computing*, 17(3):267–287, 2002.
- Michael A. Covington and Joe D. McFall. Cutting the Gordian knot: The moving-average type–token ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2):94–100, 2010.
- David K. Elson, Nicholas Dames, and Kathleen R. McKeown. Extracting social networks from literary fiction. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics*, pages 138–147, Uppsala, Sweden, 2010.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in Python, 2020.
- John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- Fotis Jannidis and Gerhard Lauer. Burrows’s Delta and its use in German literary history. In Matt Erlin and Lynne Tatlock, editors, *Distant Readings: Topologies of German Culture in the Long Nineteenth Century*, pages 29–54. Camden House, Rochester, NY, 2014.
- William H. Kruskal and W. Allen Wallis. Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association*, 47(260):583–621, 1952.
- Philip M. McCarthy and Scott Jarvis. MTL D, vocd-D, and HD-D: A validation study of sophisticated approaches to lexical diversity assessment. *Behavior Research Methods*, 42(2):381–392, 2010.

- Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.
- Frederick Mosteller and David L. Wallace. *Inference and Disputed Authorship: The Federalist*. Addison-Wesley, Reading, MA, 1964.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Project Gutenberg. Project Gutenberg. <https://www.gutenberg.org>, 2026. Catalog file `pg_catalog.csv` and plain-text ebooks, accessed September 2026.
- Christof Schöch, Tomaž Erjavec, Roxana Patras, and Diana Santos. Creating the European literary text collection (ELTeC): Challenges and perspectives. *Modern Languages Open*, 1:25, 2021.
- Efstathios Stamatatos. A survey of modern authorship attribution methods. *Journal of the American Society for Information Science and Technology*, 60(3):538–556, 2009.
- Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

A Reproducing the results

```

pip install -r requirements.txt
python -m spacy download de_core_news_lg
python main.py                # all steps
python main.py --step analyse # only the analysis (uses features.csv)
python -m pytest               # unit tests

```